Trajuan Smith **ID:** 002783094 **Section:** 002

Course: CSC 4370 — Web Development
Assignment: Homework 3 — Chez Session
Enrollment Level: Undergraduate
Submission Date: November 6, 2026

1. Project Links

Live Website (Codd): https://codd.cs.gsu.edu/~tsmith413/WebDev/HW/hw3/index.php

GitHub Repository: https://github.com/Myriad21/HW3_Bistro_amnesia

Course Homepage: https://codd.cs.gsu.edu/~tsmith413/WebDev/HW/hw1/index.html

2. Implementation Explanation

2.1 Form Submission and Session Storage

The ordering process begins in `index.php`, where the HTML form collects the customer's selections. When the user clicks **Send to Kitchen**, the form sends the selected values to `process_order.php` using the POST method.

The `process_order.php` file includes `order_functions.php` and calls the `order()` function. This function retrieves the submitted values from `$_POST` and stores them in three session variables:

- `$_SESSION['dessert']` — stores the selected dessert.
- `$_SESSION['drink']` — stores the selected drink.
- `$_SESSION['drinkSize']` — stores the selected drink size.

The session is initialized using `session_start()` in `order_functions.php`, allowing these values to remain available across multiple page requests. This makes it possible to display the customer's order on the receipt page and restore their selections when they return to the form.

2.2 Receipt Generation and Missing-Selection Handling

After the form is submitted, `process_order.php` displays the order summary generated by the `order()` function. The receipt shows the customer's selected dessert, drink, and drink size.

It also displays additional information generated by the helper functions in `order_functions.php`, including the order completion status, suggested pairing, estimated total, and order profile.

The application also handles incomplete submissions by providing fallback values for missing selections. The `order()` function uses the null coalescing operator (`??`) when accessing `$_POST`, preventing undefined-array-key warnings if a field is not submitted. Missing or unselected values are stored and displayed as `(not selected)`.

The pricing functions calculate the estimated total based on the available selections. Unselected items contribute $0.00 to the total, allowing the application to produce a receipt even when the order is incomplete.

For example, when a customer selects Hot Chocolate and a Small drink size without selecting a dessert, the receipt displays `(not selected)` for the dessert, reports that 2 of 3 choices are complete, and calculates an estimated total of $2.75. The application also displays a message prompting the customer to select both a dessert and a drink to receive a pairing suggestion.

This fallback behavior allows the application to handle incomplete orders without interrupting the ordering process.

2.3 Session Restoration (Order Again)

After viewing their receipt, customers can click the **Order Again** button to return to `index.php`. This navigation does not clear the PHP session, allowing the customer's previous selections to remain available.

When `index.php` loads, it uses three restoration helper functions from `order_functions.php`:

- `dessertSelected()` — compares the dessert option against the saved session value and returns `selected` when they match.
- `drinkSelected()` — compares the drink option against the saved session value and returns `selected` when they match.
- `drinkSizeSelected()` — compares the drink-size option against the saved session value and returns `checked` when they match.

These functions restore the previous selections in the dropdown menus and radio buttons without requiring the customer to enter them again.

For example, when an incomplete order containing Hot Chocolate and a Small drink size is submitted, clicking **Order Again** returns to the form with Hot Chocolate still selected and the

Small radio button checked. The dessert dropdown remains on its default option because no dessert was previously selected.

This demonstrates how PHP sessions preserve information across separate HTTP requests and improve the ordering experience by maintaining the customer's progress.

2.4 Session Reset (Forget Order)

The **Forget Order** feature allows customers to clear their previous selections and start a new order. When the customer clicks the Forget Order button, the browser navigates to `forget_order.php`.

This file begins by calling `session_start()` to access the existing session. It then uses `unset()` to remove the three stored order variables: `$_SESSION['dessert']`, `$_SESSION['drink']`, and `$_SESSION['drinkSize']`. After removing these variables, `session_destroy()` is called to destroy the stored session data.

Finally, the `header('Location: index.php')` function sends a redirect response to return the customer to the order form. The `exit` statement prevents additional PHP code from executing after the redirect.

When `index.php` loads again, the restoration helper functions no longer find saved selections, so the dessert and drink dropdowns display their default options and none of the drink-size radio buttons are selected. The estimated total also returns to $0.00, the order profile displays "No selections yet," and the guidance message prompts the customer to begin selecting their order.

This reset functionality ensures that customers can start a new order without carrying over selections from a previous session.

3. Test, Validation, and Reset Evidence

Test	Steps / Input	Expected Result	Actual Result	Status
1 — Complete Order	Select Cheesecake, Coffee, Medium; submit	Complete receipt, $7.10 total	Receipt displayed correctly; $7.10	Pass
2 — Missing Selection	Leave dessert empty; select Hot Chocolate, Small; submit	2/3 choices, $2.75 total, no errors	Missing dessert handled; $2.75	Pass
3 — Order Again	Submit incomplete order; click Order Again	Previously selected values restored	Hot Chocolate and Small restored	Pass
4 — Forget Order	Click Forget Order after Test 3	Selections cleared; total resets to $0.00	Form and feedback reset correctly	Pass
5 — Cross-Browser	In Chrome, submit Cheesecake, Coffee, Medium; use Order Again and Forget Order	$7.10 receipt, restored selections, successful reset	All three functions worked correctly in Chrome	Pass

Test Environment: Local PHP development server (`localhost:8000`), Brave browser on macOS.

Browser Version: Brave 1.97.56 (Official Build) (arm64)

Validation: PHP syntax checks passed for all five PHP files. The W3C CSS/HTML Validator reported zero errors.

Figure 1 — Complete Order Submission: Cheesecake, Coffee, Medium; $7.10 total

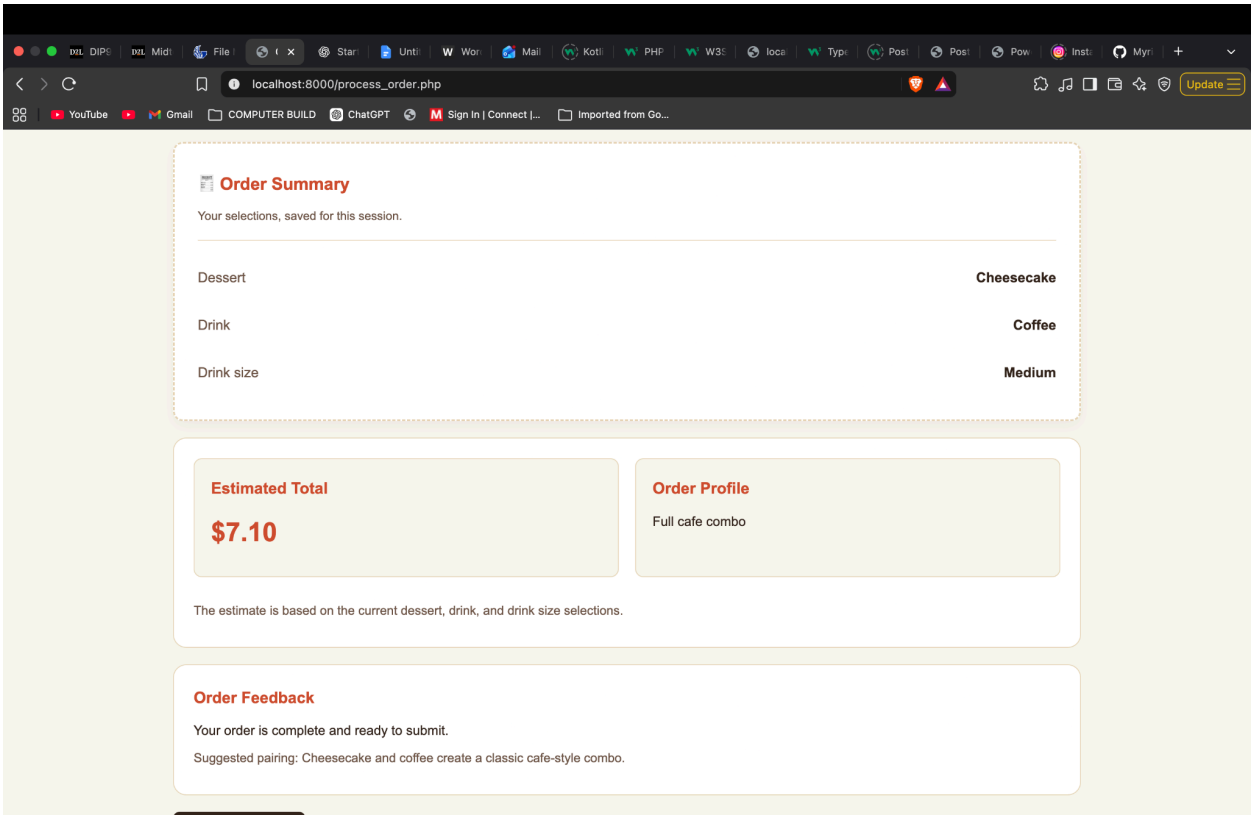


Figure 2 — Missing Selection: No dessert, Hot Chocolate, Small; $2.75 total.

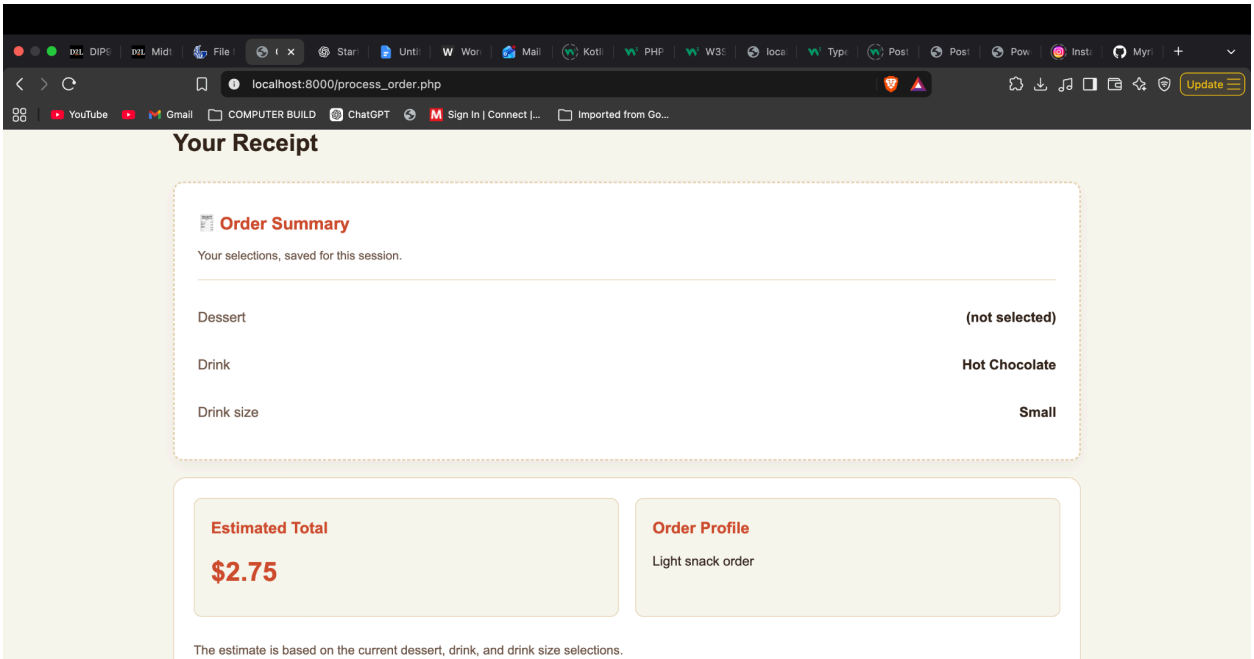


Figure 3 — Order Again: Previous drink and size selections restored.

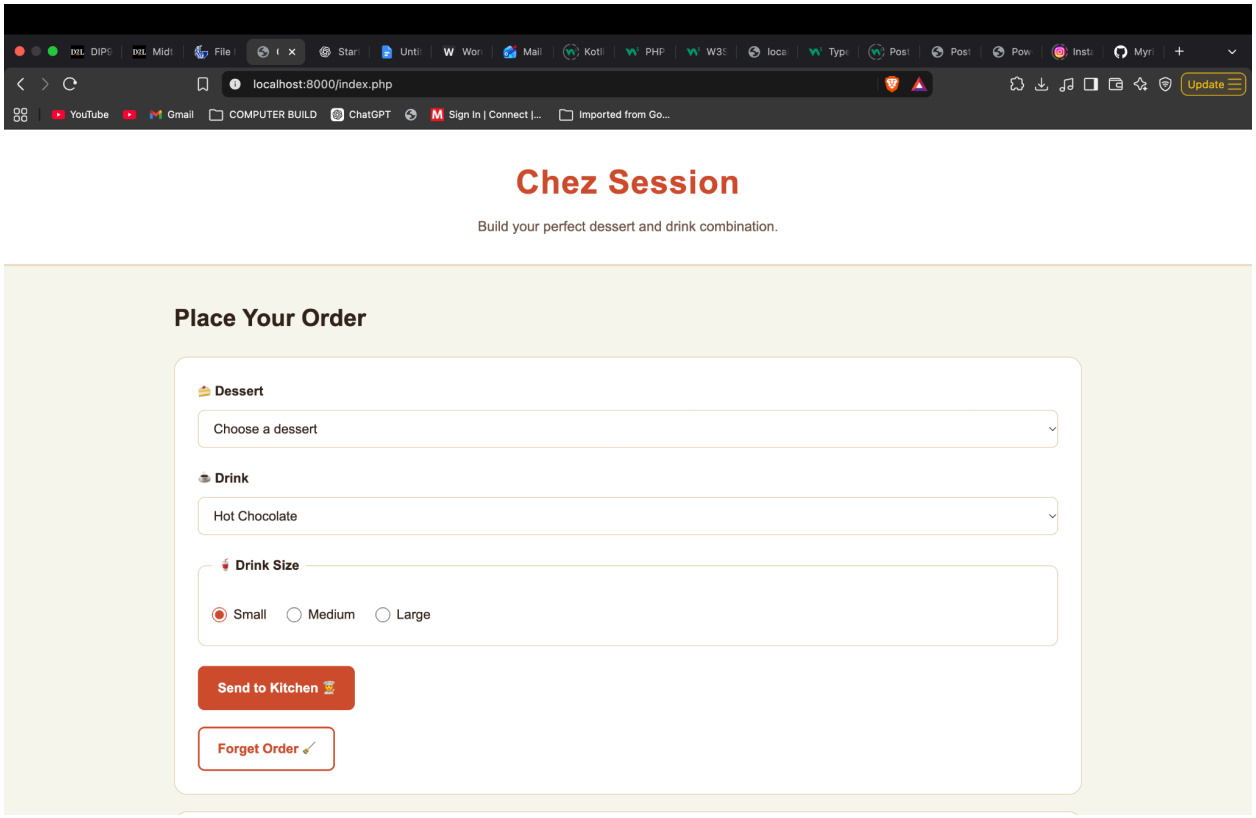


Figure 4 — Forget Order: **Selections cleared** and pricing reset.

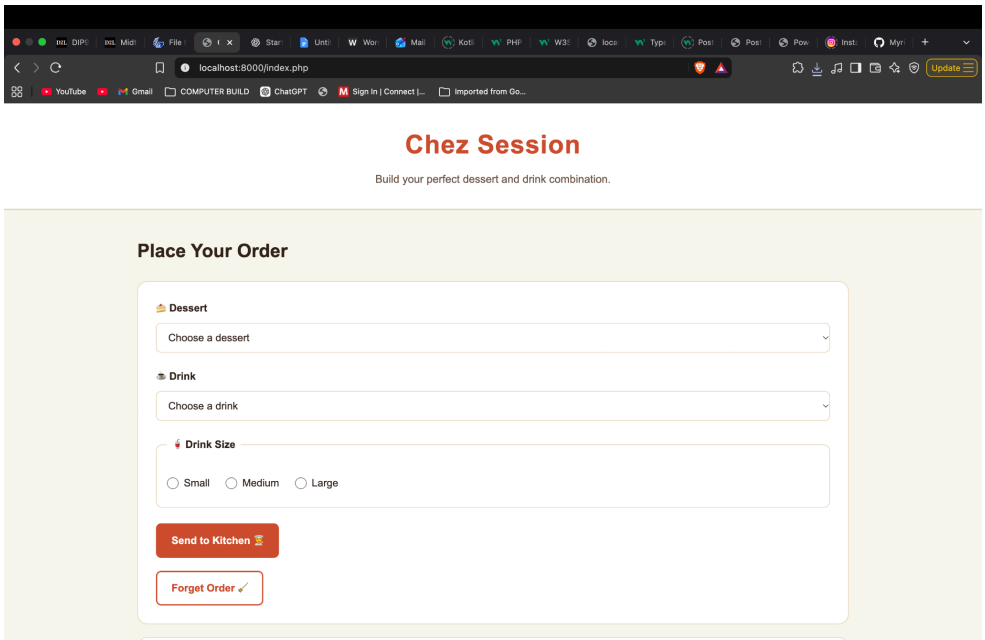
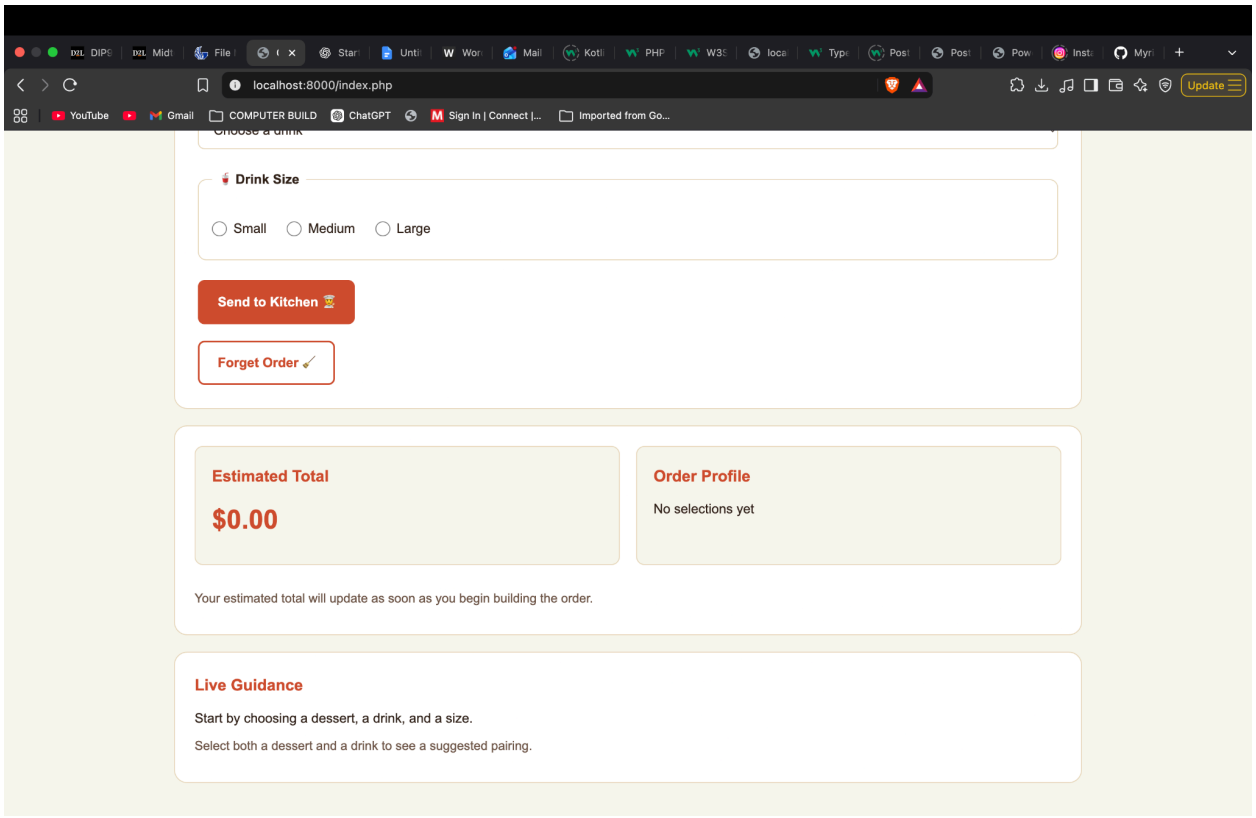


Figure 4.2 — Forget Order: Selections cleared and **pricing reset**.



3.1 Cross-Browser Testing

The Chez Session application was tested in Brave and Google Chrome on macOS using the local PHP development server at `localhost:8000`.

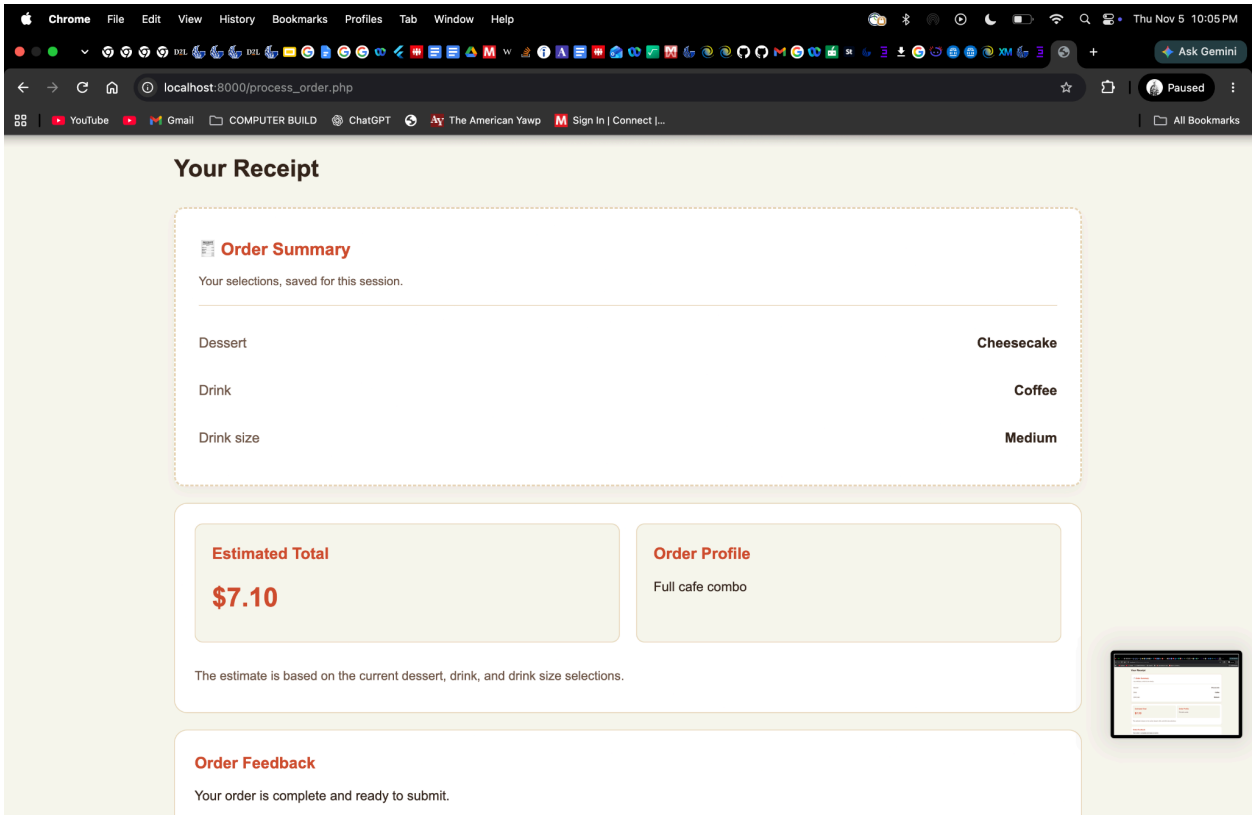
In Chrome, a complete order containing Cheesecake, Coffee, and Medium was submitted successfully. The receipt displayed the correct selections and an estimated total of $7.10.

The **Order Again** feature correctly restored all three selections when returning to the order form. The **Forget Order** feature also cleared the saved selections and returned the form to its default state.

All three Chrome tests passed, confirming that the tested ordering and session-management functionality worked consistently in both browsers.

Browser Version: Version 157.0.8103.38 (Official Build) (arm64)

Figure 5 — Cross-Browser Testing in Google Chrome:



3.2 Code Validation

The Chez Session application was checked for PHP syntax errors using the `php -l` command in the terminal. All five PHP files—`index.php`, `process_order.php`, `order_functions.php`, `forget_order.php`, and `about.php`—passed the syntax checks without errors.

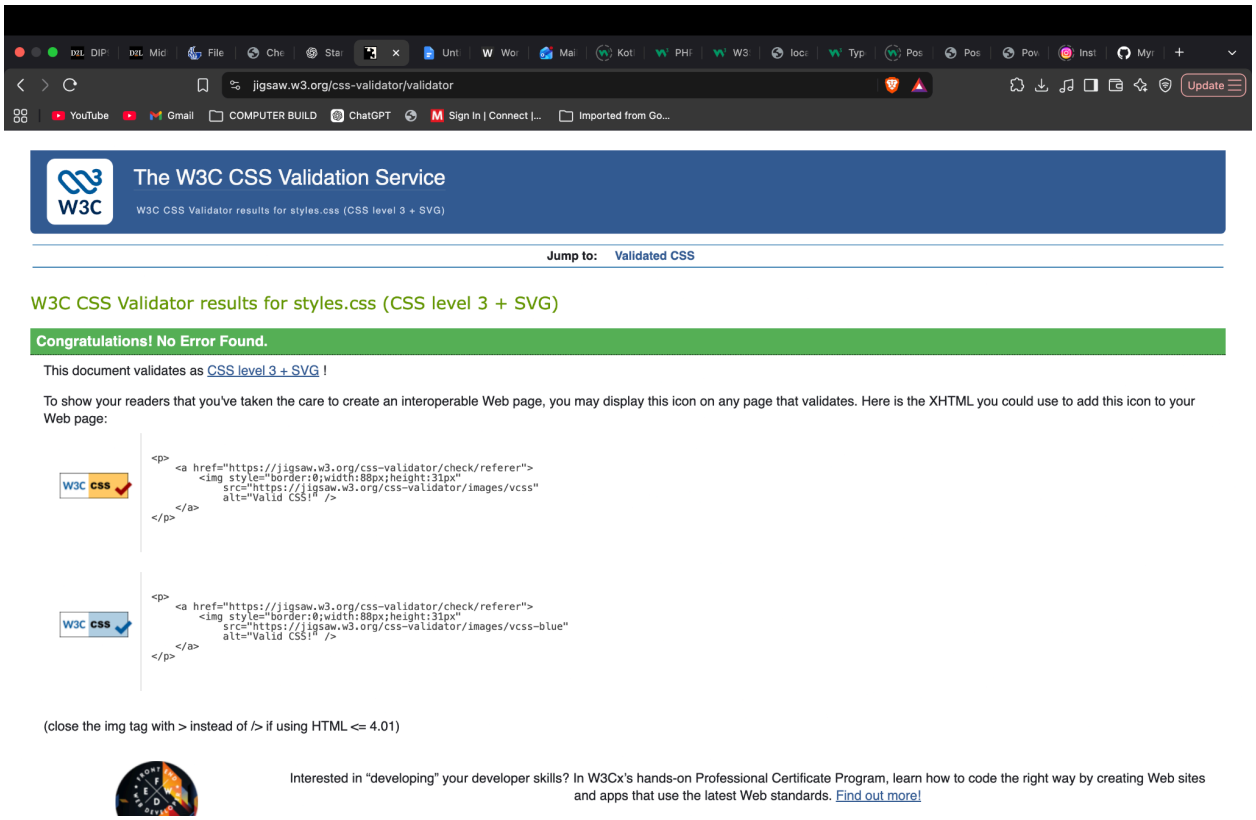
The application's stylesheet, `css/styles.css`, was also checked using the W3C CSS Validation Service. The validator reported zero CSS errors.

These checks helped verify that the PHP files contained valid syntax and that the stylesheet followed CSS validation requirements. Functional testing was performed separately to confirm that order submission, session restoration, and session resetting behaved as expected.

Figure 6 — PHP Syntax Validation

```
trae_21@Trajuans-MacBook-Pro hw3 % php -l index.php
php -l process_order.php
php -l order_functions.php
php -l forget_order.php
php -l about.php
No syntax errors detected in index.php
No syntax errors detected in process_order.php
No syntax errors detected in order_functions.php
No syntax errors detected in forget_order.php
No syntax errors detected in about.php
trae_21@Trajuans-MacBook-Pro hw3 %
```

Figure 6.1 — CSS Validation



4. Critical-Thinking Investigation

4.1 U1 — The Shared Kiosk Dilemma

Two fictional customers use the same browser profile. Save customer A's order, close the tab, reopen the same origin, and let B start ordering. What does B inherit, and what assumption about identity failed? Design a visible handoff control that removes only the lab order key while preserving quest progress. Test the handoff and a reload without a handoff. Compare this with PHP session isolation: does a shared browser necessarily mean a new session?

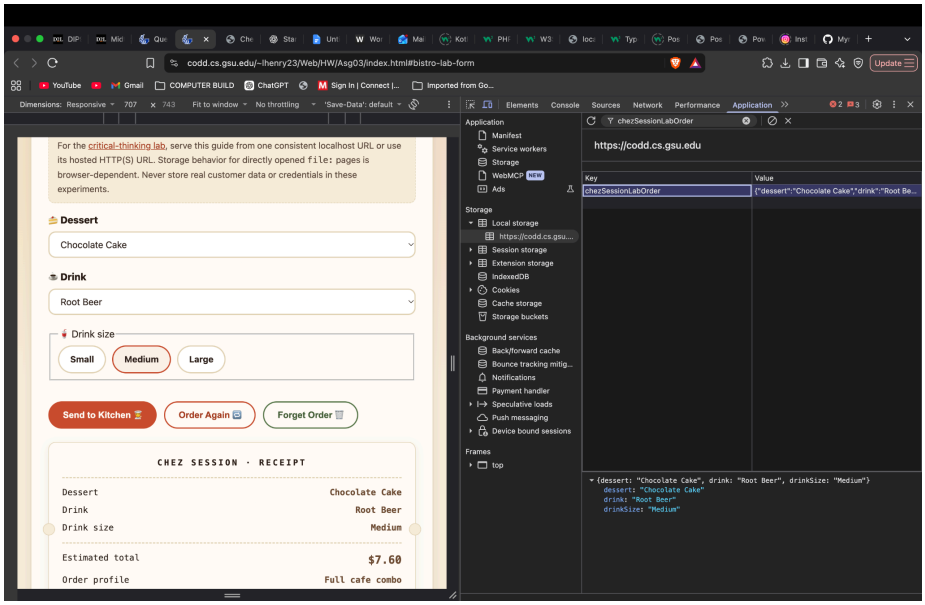
Experiment Details

- **Browser:** Brave 1.97.56 (Official Build) (arm64)
- **Date:** November 5, 2026
- **Origin:** `https://codd.cs.gsu.edu` (HTTPS, hostname `codd.cs.gsu.edu`, port 443)
- **Storage key:** `chezSessionLabOrder`

Prediction: Closing the Bistro Lab tab without clearing Local Storage would preserve Customer A's order. Customer B, using the same browser profile, would inherit those selections.

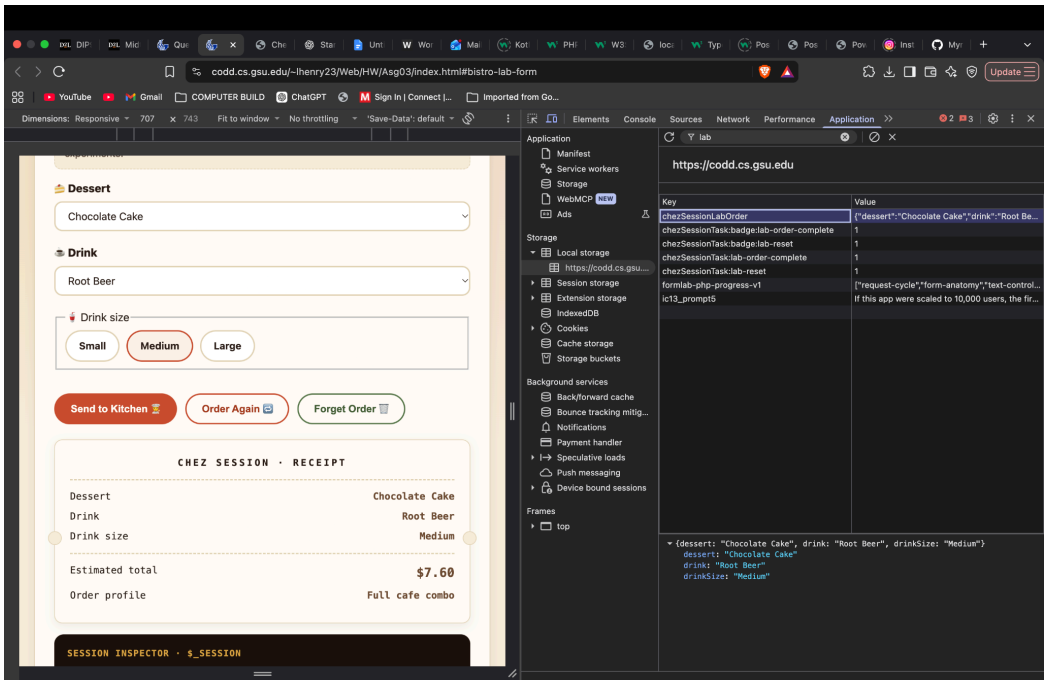
Observation 1 — Reload Without Handoff

Customer A selected Chocolate Cake, Root Beer, and Medium, then saved the order. I closed the tab and reopened the Bistro Lab using the same Brave profile. The stored value was `{"dessert":"Chocolate Cake","drink":"Root Beer","drinkSize":"Medium"}`.



Expected: The previous order would remain available.

Actual: All three selections remained, confirming that closing the tab did not clear the saved order.



Observation 2 — Explicit Handoff

I clicked Forget Order and inspected Local Storage in DevTools.

Expected: The order key would disappear, the form would reset, and unrelated website data would remain.

Actual: `chezSessionLabOrder` disappeared, all selections returned to their defaults, and the total became $0.00. Other keys, including `formlab-php-progress-v1`, remained. Reloading the page afterward confirmed that the form stayed cleared.

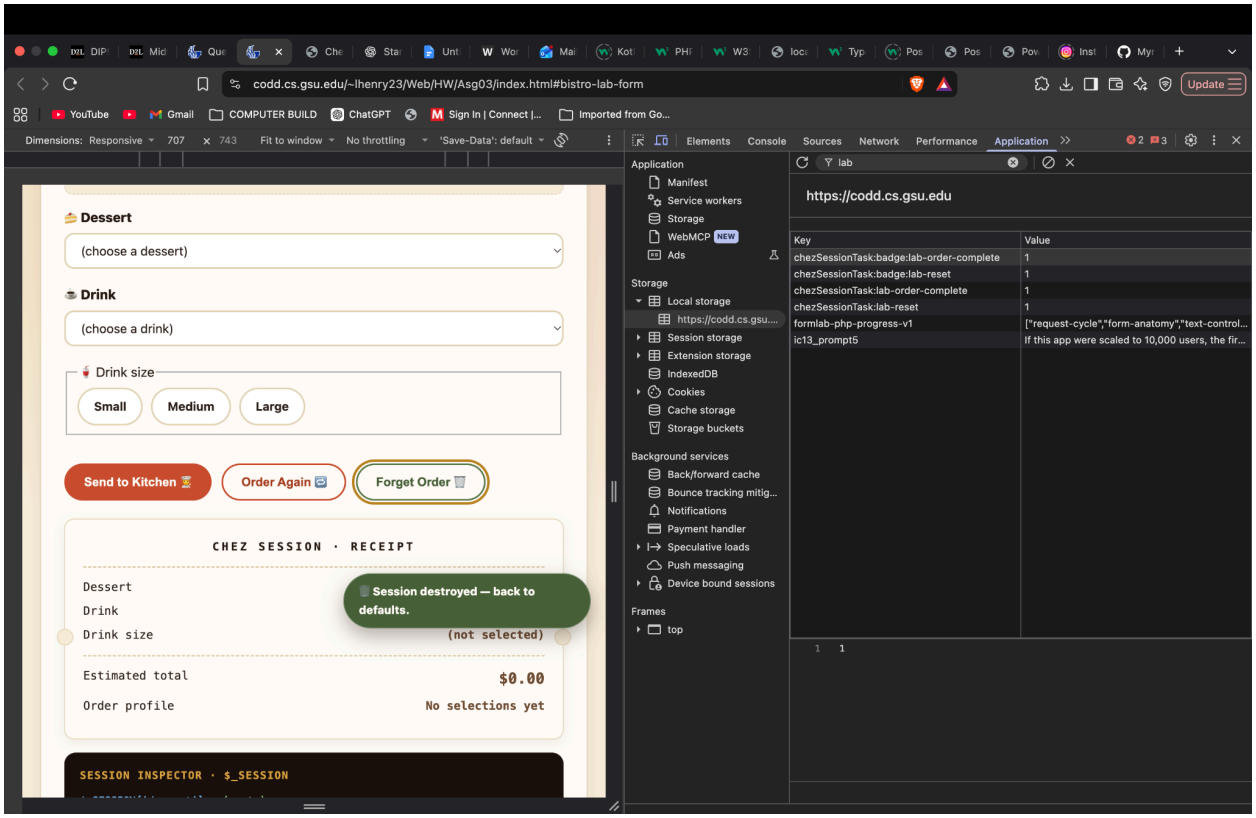


Figure 8 — U1 Handoff Test: Forget Order removes the saved order key, resets the form, and preserves unrelated Local Storage entries.

Analysis: The failed assumption was that closing a tab or changing customers creates a separate storage identity. Local Storage is associated with an origin and browser profile, not an individual customer. A competing explanation is that the browser simply restored its form controls, but the saved JSON record in DevTools provides evidence of persistent application storage. One limitation is that the experiment used a single Brave profile rather than multiple browser profiles.

Cookies differ because their persistence depends on expiration and browser behavior. PHP sessions store order data on the server, while a browser typically sends a session cookie to identify the session. A shared browser does not necessarily create a new PHP session, so Customer B could also inherit Customer A's server-side order if the session remains active.

Proposed Solution: I would add a clearly labeled Finish Customer Session button that removes only `chezSessionLabOrder`, resets the controls, and displays a confirmation that the kiosk is ready for another customer. This preserves unrelated guest progress. The trade-off is that customers must remember to use the button; an inactivity timeout could provide additional protection but might interrupt unfinished orders.

Success Test: Removing the order key resets the form and prevents restoration after reloading. This behavior was observed.

Edge-Case Test: Closing and reopening the tab without using the handoff control restores the previous order. This behavior was also observed and demonstrates why an explicit handoff is necessary.

Proposed pseudocode solution

```
function finishCustomerSession() {

    localStorage.removeItem("chezSessionLabOrder");

    resetOrderForm();

    showMessage("Ready for the next customer.");

}
```

4.2 U4 — The Vanishing Pantry

Save an order and compare a reload with explicitly removing only chezSessionLabOrder in DevTools. If available, repeat in a private window and record when its data disappears rather than assuming it. Design an honest “saved on this device” message and a recovery path when storage is unavailable or cleared. Test a normal save and a simulated write failure. What can localStorage guarantee, and what can it not guarantee about persistence or offline use

Experiment Details

- **Browser:** Brave 1.97.56 (Official Build) (arm64)
- **Date:** November 5, 2026
- **Origin:** `https://codd.cs.gsu.edu` (HTTPS, hostname `codd.cs.gsu.edu`, port 443)
- **Storage key:** `chezSessionLabOrder`

Prediction: Saving an order should create a Local Storage record that remains available after reloading the page. If the record is explicitly removed, the application should no longer be able to restore those saved selections. If saving fails because storage is unavailable, the application should not falsely report that the order was saved.

Observation 1 — Normal Save and Reload

Action: I selected Chocolate Cake, Root Beer, and Large in the Bistro Lab and clicked Send to Kitchen. After confirming that the order was saved in Local Storage, I reloaded the page.

Stored value: `{"dessert":"Chocolate Cake","drink":"Root Beer","drinkSize":"Large"}`

Expected result: The saved order would remain available after reloading, and the application would restore the previous selections.

Actual result: The `chezSessionLabOrder` key remained present, and all three selections were restored correctly. The receipt continued to display the order with an estimated total of $8.15.

Conclusion: The experiment confirmed that Local Storage preserves saved order data across page reloads within the same browser profile and origin.

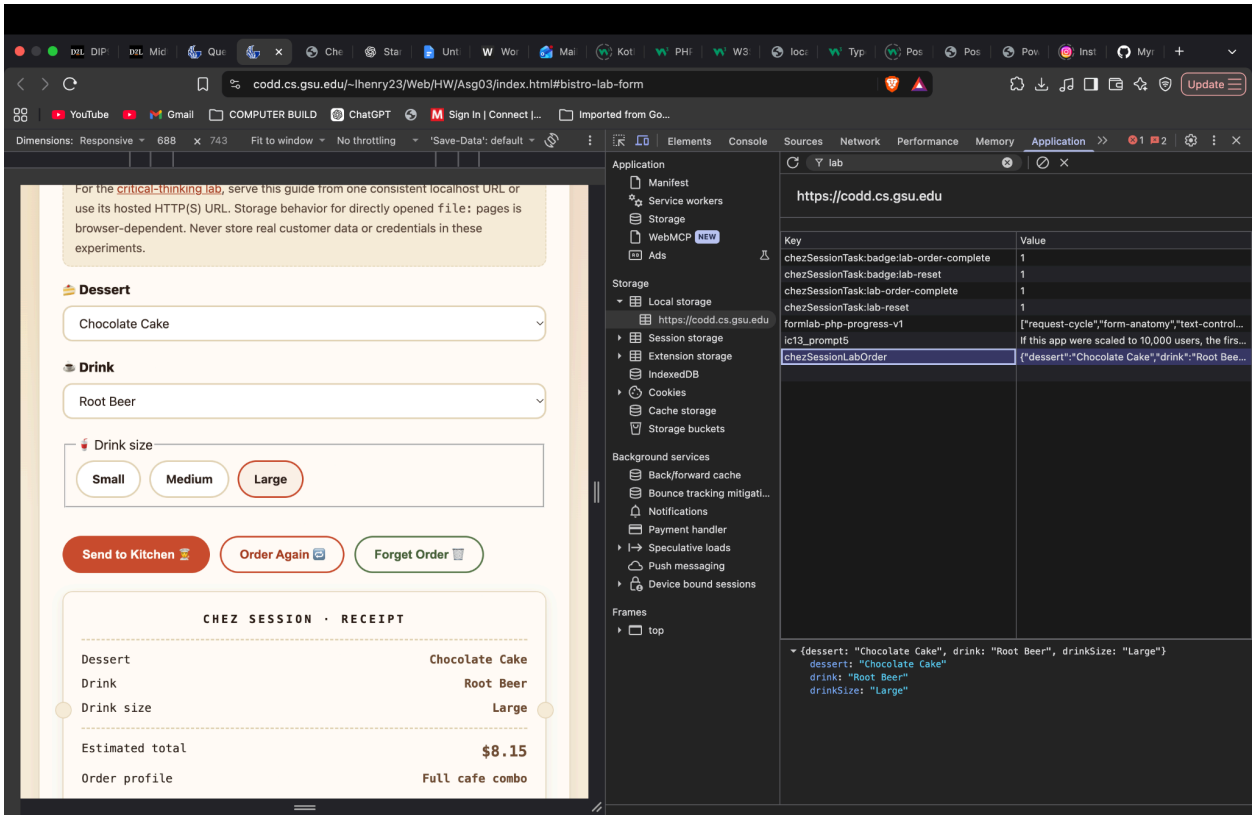


Figure 9 — U4 Normal Save and Reload: The order remains stored in Local Storage and is restored after reloading.

Observation 2 — Manual Removal of the Saved Order

Action: After saving Chocolate Cake, Root Beer, and Large, I opened Brave DevTools and manually deleted the `chezSessionLabOrder` key from Local Storage without clicking Forget Order. I inspected the form before and after reloading the page.

Stored value: The `chezSessionLab0rder` key was absent. Other Local Storage entries remained available.

Expected result: Removing the key would prevent the saved order from being restored after reloading. The currently displayed controls might retain their values until the page refreshed.

Actual result: Immediately after deleting the key, the form still displayed Chocolate Cake, Root Beer, and Large, with an estimated total of $8.15. After reloading, the dessert and drink dropdowns returned to their default options, no drink size was selected, and the estimated total became $0.00. The order key remained absent.

Conclusion: The experiment demonstrated that the visible interface and persistent storage can temporarily contain different information. Reloading caused the application to reflect the missing saved order.

Figure 10 — Before Reload: The Local Storage key is absent, but the previous selections remain visible.

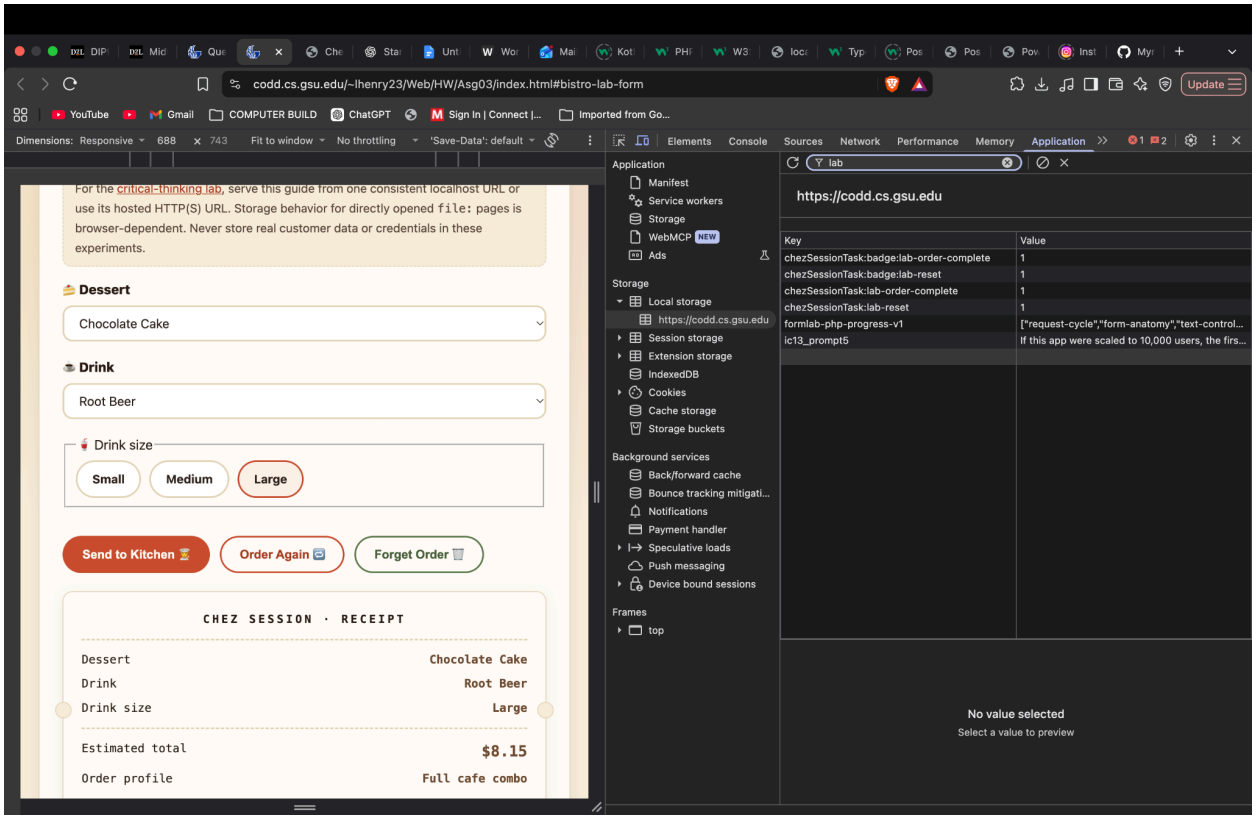
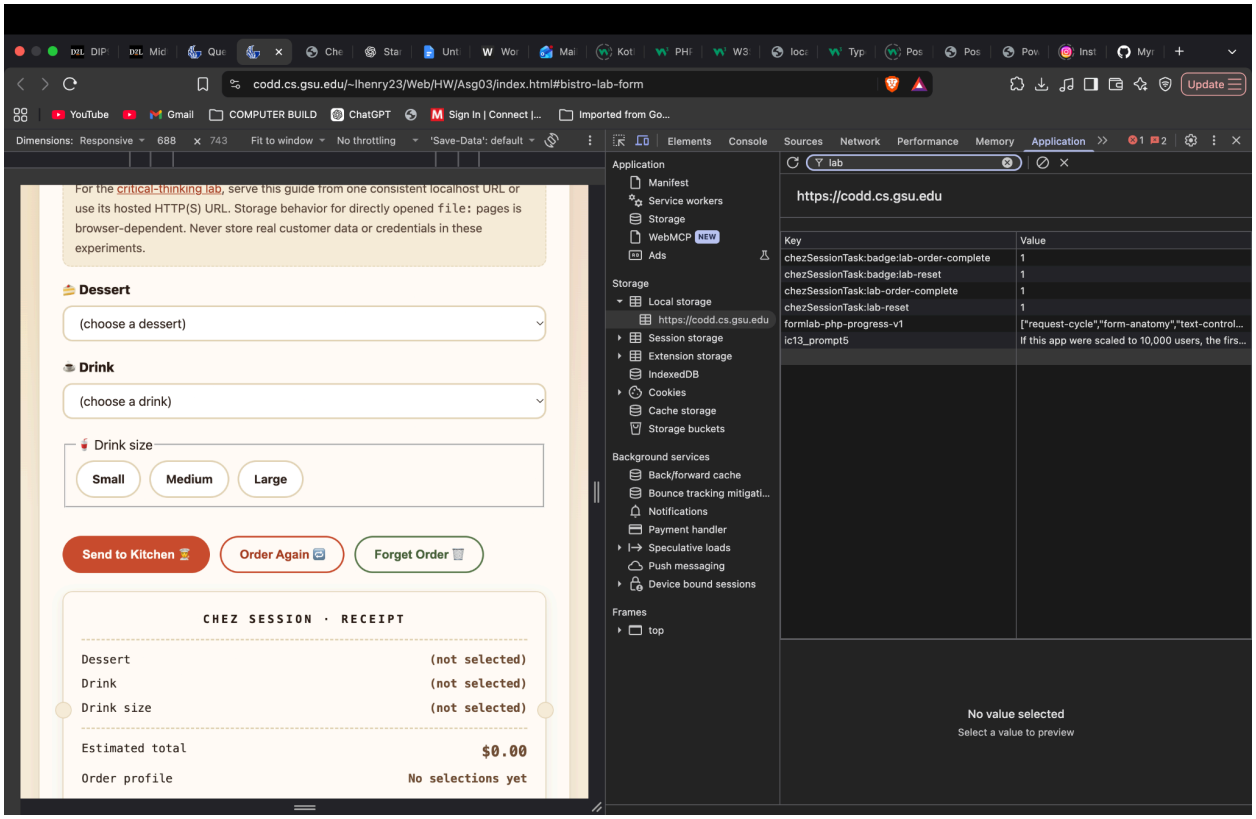


Figure 11 — After Reload: The Local Storage key remains absent, and the form has reset.



Additional Test — Simulated Local Storage Write Failure

Action: I used the Brave DevTools Console to create a simulated storage object whose `setItem()` method throws a `QuotaExceededError`. I attempted to save an invented order containing Chocolate Cake, Root Beer, and Large inside a `try...catch` block.

Expected result: The simulated write would fail, and the error handler would report the failure rather than displaying a successful-save message.

Actual result: The Console displayed `Save failed: QuotaExceededError` followed by `Message: Simulated storage write failure`. The success message was not displayed.

Conclusion: The experiment demonstrated that a storage write failure can be detected and reported using exception handling. This was a controlled simulation rather than a failure produced by the actual Bistro Lab application. A production implementation should display a visible warning when saving fails instead of implying that the order was successfully preserved.

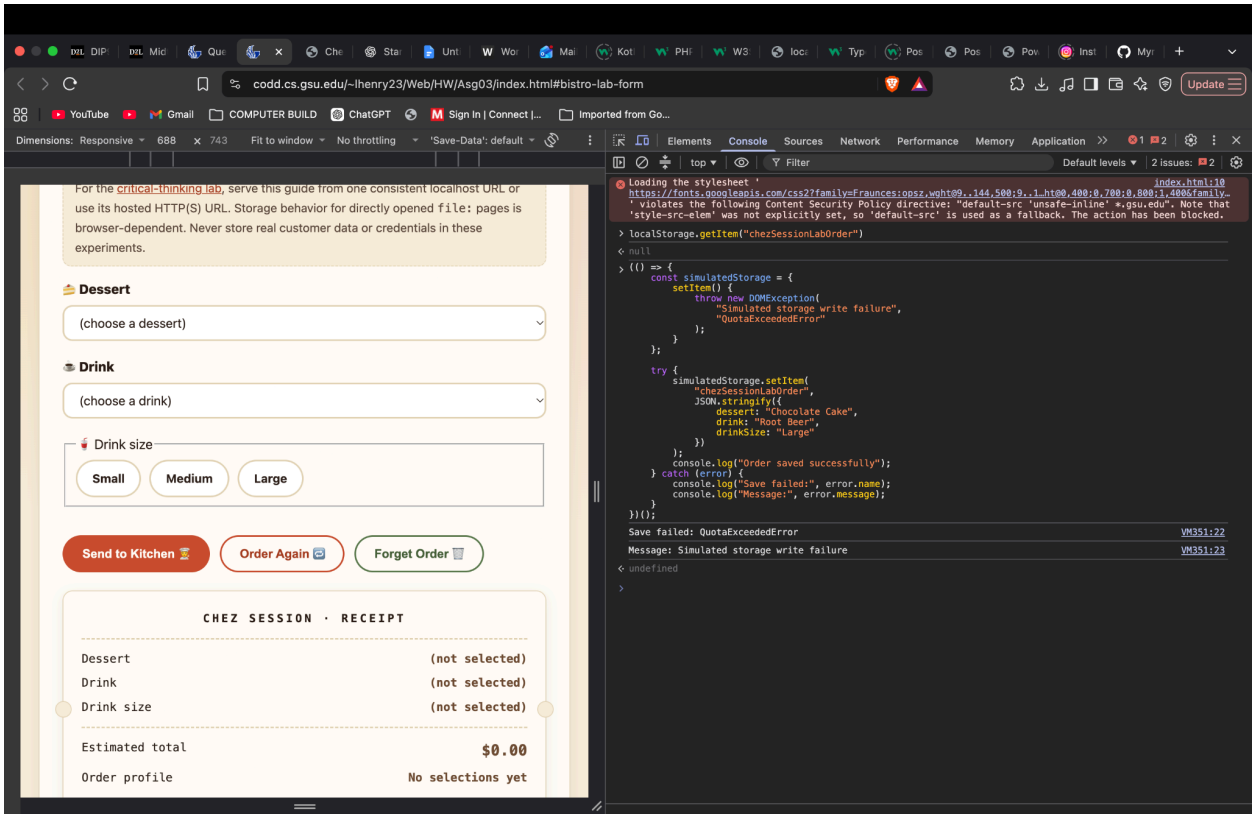


Figure 12 — U4 Simulated Storage Failure: The Console catches a deliberately generated `QuotaExceededError` and reports that the save failed.

Proposed pseudocode

```
function saveOrder(order) {
    try {
        localStorage.setItem(
            "chezSessionLabOrder",
            JSON.stringify(order)
        );

        showStatus("Order saved on this device.");
        return true;
```

```
    } catch (error) {
        showStatus(
            "We couldn't save your order. Please try again."
        );
        return false;
    }
}
```

Analysis and Limitations

The experiments showed that Local Storage can preserve an order across page reloads, but it does not guarantee permanent storage. When I manually deleted the saved order, the displayed selections initially remained unchanged. After reloading, the form reset because the saved record was missing. This demonstrates that the visible interface does not always reflect the current storage contents.

A competing explanation is that the browser was remembering the form selections independently. However, the Local Storage inspection confirmed that the saved record had been removed, while the interface still displayed the previous values. A limitation of this investigation is that the write failure was simulated rather than caused by an actual browser storage restriction.

Unlike Local Storage, cookies can be sent to the server with HTTP requests. PHP sessions store order information on the server and typically use a session cookie to identify the customer. Our PHP application uses `$_SESSION` rather than Local Storage to restore selections.

Local Storage supports saving data without an immediate server request, but it cannot guarantee that data will survive browser clearing, storage restrictions, or changes in browser profiles. It also does not guarantee that the entire website will work offline.

My proposed recovery design displays a success message only after a successful write and a warning when saving fails. The trade-off is that the application must handle missing or unavailable storage rather than assuming every order can be recovered.

5. Reflection

One limitation of my Chez Session application is that it only maintains the customer's current order through PHP sessions. Although the session allows selections to persist between pages,

the order information is lost when the customer uses Forget Order or when the session expires. The application does not maintain a permanent history of previously submitted orders.

One improvement I would implement next is an order history feature using a database. Instead of relying exclusively on PHP sessions, completed orders could be saved with an order identifier and timestamp. Customers could then review previous orders without interfering with the current session. This would make the application more useful while preserving the existing Order Again and Forget Order functionality.

6. References and Tool-Use Disclosure

6.1 References

- Georgia State University. *CSC 4370 — Homework 3: Chez Session.* Assignment instructions, critical-thinking lab, and submission requirements.
- MDN Web Docs. *Window: localStorage property.* https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage
- PHP Documentation. *Sessions.* https://www.php.net/manual/en/book.session.php

6.2 Tool-Use Disclosure

I used Visual Studio Code to develop and edit my PHP, HTML, and CSS files. I used Brave and Google Chrome to test the application, including browser DevTools for inspecting Local Storage and performing the critical-thinking experiments. I used PHP's command-line syntax checker to verify my PHP files and the W3C CSS Validator to check my stylesheet.

I also used ChatGPT to assist with understanding assignment requirements, explaining PHP sessions, browser storage, and troubleshooting. I independently ran the application, performed the browser experiments, inspected the results, and captured the screenshots included in this report.